

Clase 4

6 de abril de 2026

Esquema Encriptación (Cífrado)  Provee Confidencialidad  No detecta alteraciones en mensaje
No provee
INTEGRIDAD/AUTENTICACIÓN

MAC

Message Authentication Codes

Esquema MAC (Gen, Mac, Vrfy)

- Gen: $k \leftarrow \text{Gen}(1^n)$
 - Mac: $t \leftarrow \text{Mac}_k(m)$
 - Vrfy: $b := \text{Vrfy}_k(m, t)$
- emisor envía $\langle m, t \rangle$
 $b = 1$ si es válido

Seguridad de MAC

Experimento **MAC – Forge**_{A,π}(*n*)

- 1) $k \leftarrow \text{Gen}(n)$
- 2) El adversario *A* recibe 1^n y acceso al oráculo $\text{Mac}_k(.)$
Sea Q , conjunto de todas las preguntas que hace *A* al oráculo $\text{Mac}_k(.)$
El adversario *A* emite un par $\langle m, t \rangle$ de igual longitud
- 3) La salida del experimento es 1 (ÉXITO) si y sólo si:
 - (1) $\text{vrfy}(m, t) = 1$ y
 - (2) $m \notin Q$

MAC es seguro (infalsificable) ante un ataque de mensaje elegido si y sólo si:

$$\Pr[\text{MAC – Forge}_{A,\pi}(n) = 1] \leq \text{negl}(\quad)$$



MAC no protege contra ataques de REPLAY

→ N° de secuencia

→ Timestamps

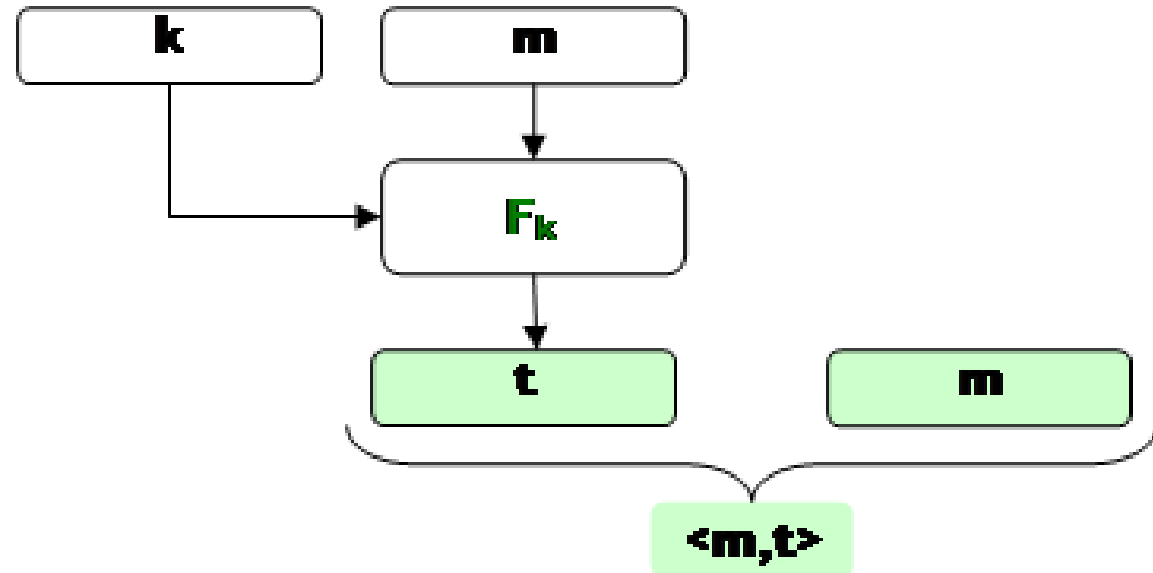
Construcción de MAC seguros para mensajes de longitud fija n

Esquema MAC (Gen, Mac, Vrfy)

- **Gen**: $k \leftarrow \{0,1\}^n$
- **Mac**: $t \leftarrow F_k(m) \mid |m| = |k| = n$
- **Vrfy**: $b := \text{Vrfy}_k(m, t)$
 - Si $|m| \neq |k| \Rightarrow 0$
 - Sino, si $F_k(m) = t \Rightarrow 1$

F_k Función
Seudoaleatoria

MAC seguro PARA MENSAJES DE
LONGITUD FIJA n



Construcción de MAC seguros para mensajes de longitud variable

Sugerencia 1:

XOR entre los bloques, autenticar el resultado: $t := \text{Mac}'_k(\oplus_i m_i)$
y emitir $\langle m_1, m_2, \dots, m_d; t \rangle$

❌ El adversario puede falsificar un tag válido sobre un nuevo mensaje, cambiando el mensaje original como para que el XOR de los bloques no cambie.

Sugerencia 2:

Autenticar cada bloque por separado: $t_i := \text{Mac}'_k(m_i)$
y emitir $\langle m_1, m_2, \dots, m_d; t_1, t_2, \dots, t_d \rangle$

❌ El adversario puede cambiar el orden de los bloques y calcular un tag válido sobre ellos (ej. $m_d, \dots, m_2, m_1$)

Sugerencia 3:

Autenticar cada bloque junto con número de secuencia: $t_i := \text{Mac}'_k(i \parallel m_i)$
y emitir $\langle m_1, m_2, \dots, m_d; t_1, t_2, \dots, t_d \rangle$

❌ El adversario puede mezclar bloques de diferentes mensajes
Ej: de $\langle m_1, m_2, \dots, m_d; t_1, t_2, \dots, t_d \rangle$ y $\langle m'_1, m'_2, \dots, m'_d; t'_1, t'_2, \dots, t'_d \rangle$ es válido $\langle m_1, m'_2, m_3, m'_4, t_1, t'_2, t_3, t'_4, \dots \rangle$

A partir de MAC (Gen' , Mac' , Vrfy') para mensajes de longitud n .

▪ **Gen'**: Gen'

▪ **Mac'**: $|m| = L < 2^{n/4}$ $|k| = n$

El mensaje m se parte en d bloques de longitud $n/4$
(se completa con ceros)

elegir $r \leftarrow \{0,1\}^{n/4}$

$t_1 = \text{Mac}'_k(r \parallel L \parallel 1 \parallel m_1)$

$t_2 = \text{Mac}'_k(r \parallel L \parallel 2 \parallel m_2)$

...

$t_d = \text{Mac}'_k(r \parallel L \parallel d \parallel m_d)$

$\Rightarrow$ emite $\langle r, t_1, t_2, \dots, t_d \rangle$



¡Ineficiente!



CBC-MAC

CBC-MAC

CBC-MAC (Gen, Mac, Vrfy)

▪ Gen: $K \leftarrow \{0,1\}^n$

▪ Mac:

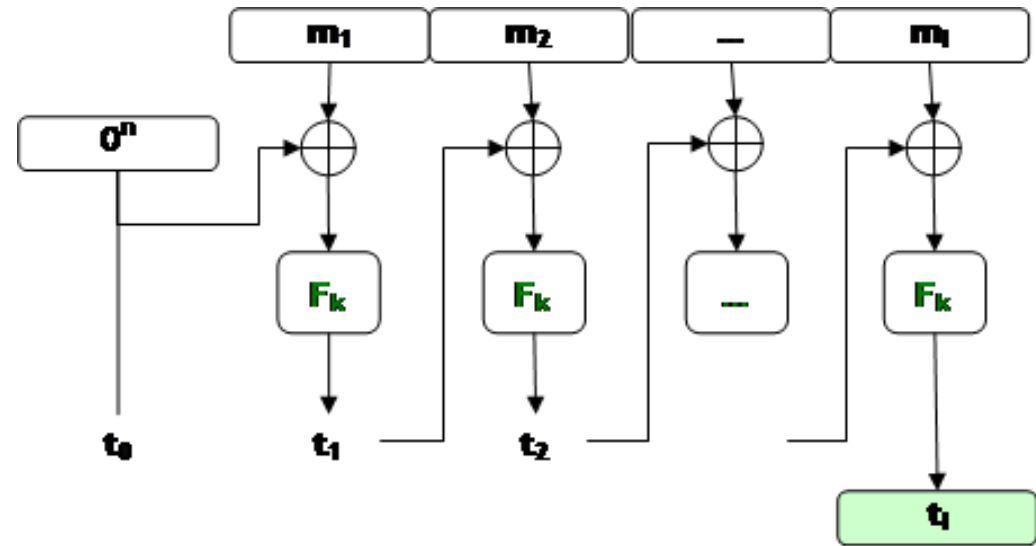
○ $|K| = n$

○ $|m| = L(n) \cdot n$

El mensaje m se parte en l bloques de longitud n

$t_0 = 0^n$

$t_i = F_k(t_{i-1} \oplus m_i) \Rightarrow$ emite t_i



La construcción anterior es infalsificable SÓLO si se permiten mensajes de una misma longitud.

Opciones seguras para CBC-MAC de distintas longitudes:

Opción 1: $k_l := F_k(|m|)$ y $t \leftarrow \text{CBC-MAC}_{k_l}(m)$

Opción 2: $m' := |m| \parallel m$ y $t \leftarrow \text{CBC-MAC}_k(m')$

Opción 3:

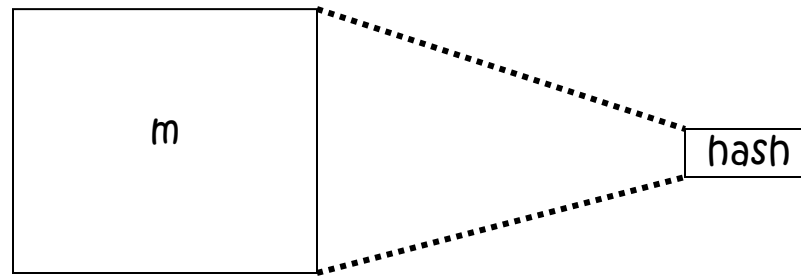
$k_1 \leftarrow \{0,1\}^n$

$k_2 \leftarrow \{0,1\}^n$

$t \leftarrow \text{CBC-MAC}_{k_1}(m)$

$\hat{t} \leftarrow F_{k_2}(t)$

Función de HASH



comprime la entrada a una longitud fija.

Permiten construir Esquemas Mac seguros.

Colisiones

Para $x \neq x'$ resulta $H(x) = H(x')$

¡Colisión!



Las colisiones van a existir

➡ Nociones de seguridad más débiles:

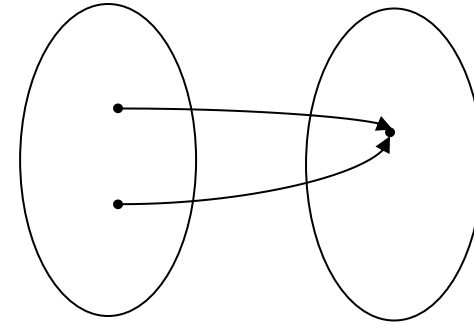
Resistente a segundas
preimágenes

Resistente a preimagen

Niveles de Seguridad

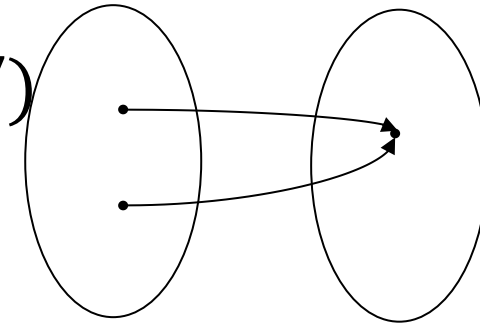
1. Resistente a Colisiones

Encontrar $x \neq x'$ tal que $H^s(x) = H^s(x')$



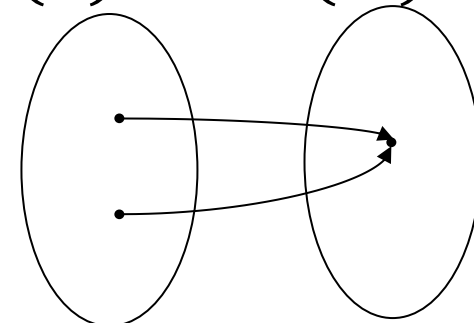
2. Resistente a Segundas Preímagenes

Dado x , encontrar $x' \neq x$ tal que $H^s(x) = H^s(x')$



3. Resistente a Preimagen

Dado $H^s(x) = y$, encontrar algún x' tal que $H^s(x') = y$



Tamaño de Salida



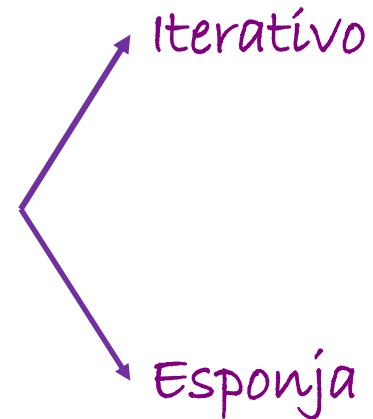
Para ser segura, una función de hash resistente a colisiones necesita tener una salida que sea mayor de 160 bits (2^{80} cálculos).

Es una condición necesaria pero no suficiente.

Primitivas de Hash

Primitiva de Hash	Salida
MD5	128 bits
Sha1	160 bits
Sha3 (Estándar actual)	224/256/384/512

Modelos de
Aplicación
(Construcciones)



CBC-MAC/Merkle

Keccak (Sha3)

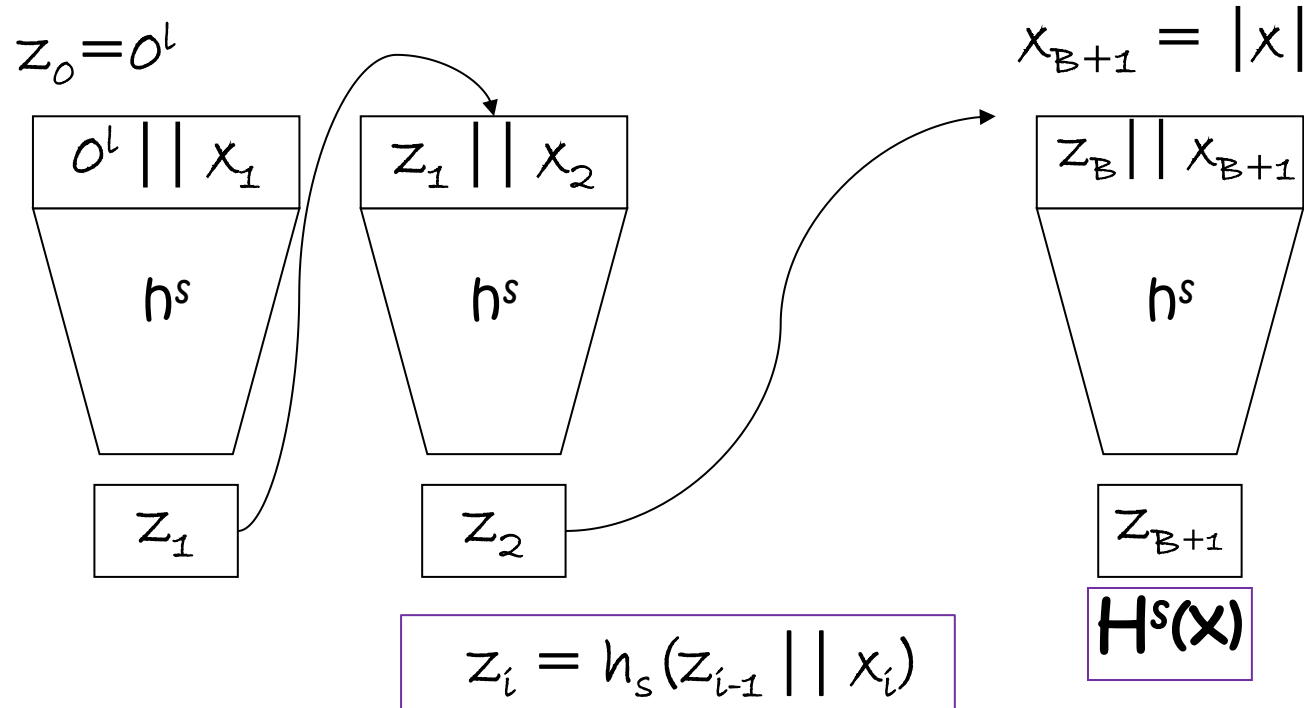
$H^s(x)$ Resistentes a Colisiones en la práctica

Transformación de Merkle Damgard

para mensajes de longitud variable.

$$|x| = L < 2^{l(n)} \quad x = x_1 x_2 x_3 \dots x_B \quad |x_i| = l$$

Se usa: $h^s: \{0,1\}^{2l} \rightarrow \{0,1\}^l$ segura



una colisión en H^s sólo puede ocurrir si hay una colisión en h^s .

NMAC (Nested Mac)

- Gen: emit (s, k_1, k_2)

$$|k_1| = |k_2| = n$$

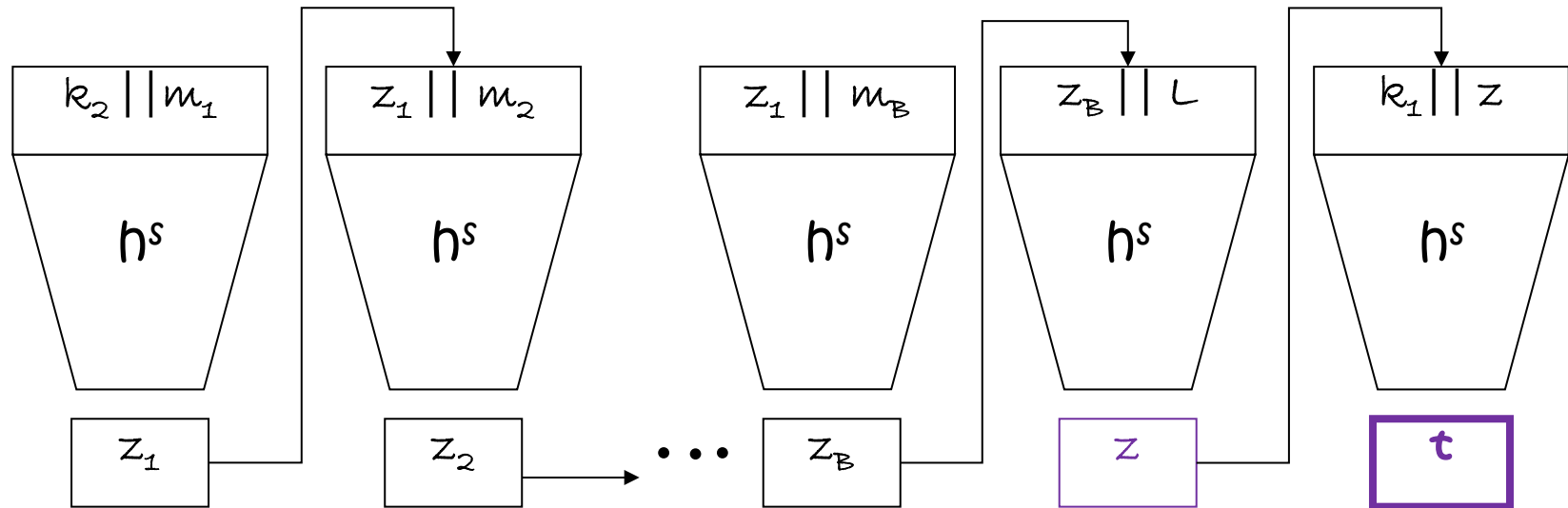
- Mac: emit $\langle m, t \rangle$

$$m = m_1 m_2 m_3 \dots m_B$$

$$|m| = L$$

$$|m_B| = n$$

$$t := h_{k_1}^s(h_{k_2}^s(m))$$



$$H_{k_2}^s(m) = z$$

$$h_{k_1}^s(z) = t$$

HMAC hash-based mac

- Gen: emite (s, k)

$$|k| = n$$

- Mac: emite $\langle m, t \rangle$

$$m = m_1 m_2 m_3 \dots m_B$$

$$|m| = L$$

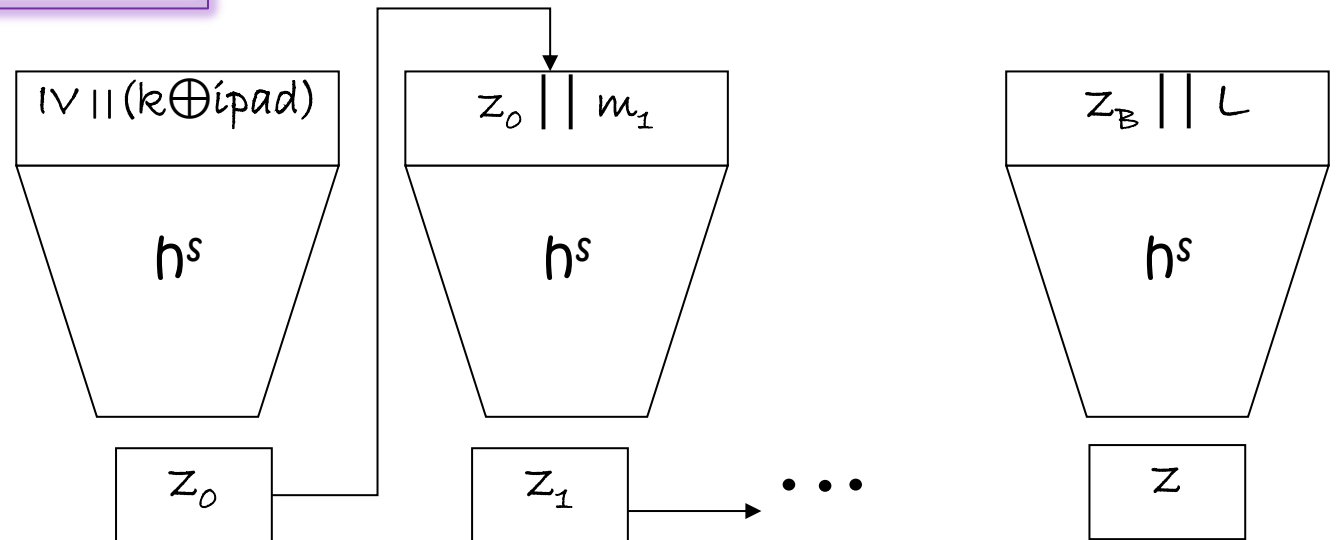
$$|m_B| = n$$

opad = n veces byte "0x36"

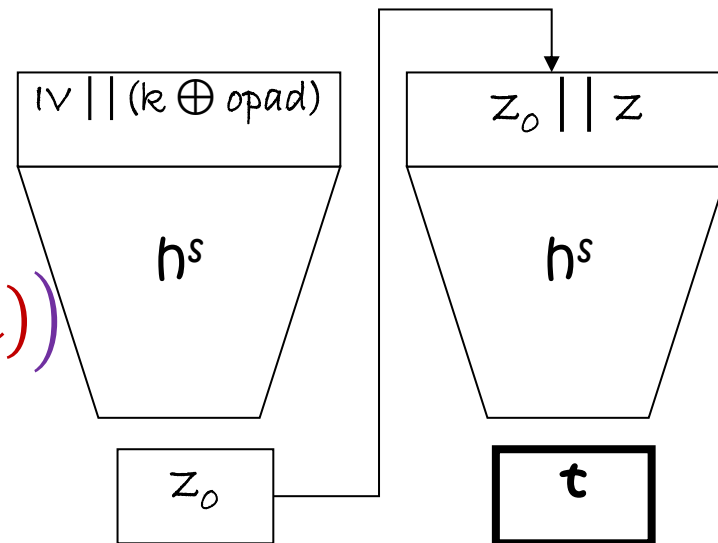
ipad = n veces byte "0x5C"

IV = constante fija de longitud n

$$t := H_{IV}^s((k \oplus opad) || H_{IV}^s((k \oplus ipad) || m))$$



$$H_{IV}^s((k \oplus ipad) || m) = z$$



$$H_{IV}^s((k \oplus opad) || z) = t$$

Cifrado autenticado

(Secure message transmission)

Privacidad + integridad

CCA-Secure + MAC-forge Secure

CCA-Secure = Seguro frente a Chosen Ciphertext Attack

Experimento $\text{PrivK}_{A,\pi}^{\text{CCA}}(n)$ para esquema de encriptación $\pi = (\text{Gen}, \text{Enc}, \text{Dec})$

- 1) $k \leftarrow \text{Gen}(n)$
- 2) El adversario A recibe 1^n y acceso al oráculo $\text{Enc}_k(.)$ y $\text{Dec}_k(.)$ y emite un par de mensajes m_0 y m_1 , de igual longitud.
- 3) Se elige un bit aleatorio $b \leftarrow \{0,1\}$.
Se calcula un cifrado $c \leftarrow \text{Enc}_k(m_b)$ y se lo entrega a A . (challenge ciphertext)
- 4) El adversario A sigue teniendo acceso a través del oráculo a $\text{Enc}_k(.)$ y $\text{Dec}_k(.)$ pero no puede preguntar sobre el último c .
 A emite un bit b' .
- 3) Si $b' = b$, la salida del experimento es 1 (ÉXITO). Sino, es 0.

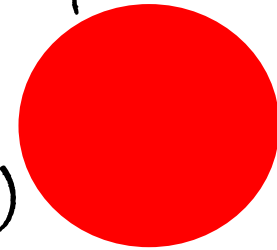
El esquema de encriptación $\pi = (\text{Gen}, \text{Enc}, \text{Dec})$ es seguro ante un ataque de **cifrado elegido** sí y sólo sí, para todo adversario PPT A :

$$\Pr[\text{PrivK}_{A,\pi}^{\text{CCA}}(n) = 1] \leq \frac{1}{2} + \text{negl}(n)$$

Se podrían combinar cifrado y autenticación de 3 formas:

1. cifrar y autenticar por separado.

Emíte $\langle c, t \rangle$, donde $c \leftarrow \text{Enc}_{k_1}(m)$, $t \leftarrow \text{Mac}_{k_2}(m)$

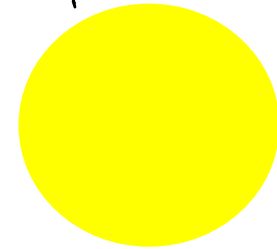


INSEGURO.

La etiqueta t puede brindar información de m .

2. autenticar, y luego cifrar.

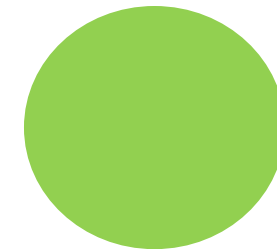
Emíte c , donde $c \leftarrow \text{Enc}_{k_1}(m \parallel \text{Mac}_{k_2}(m))$



Puede ser seguro, requiere prueba de seguridad. (x padding)

3. cifrar, y luego autenticar.

Emíte $\langle c, t \rangle$, donde $c \leftarrow \text{Enc}_{k_1}(m)$, $t \leftarrow \text{Mac}_{k_2}(c)$



SEGURO.

siempre que los algoritmos de encriptación y mac lo sean y claves independientes.

Construcción de cifrado autenticado seguro:

$\pi_E(\text{Gen}_E, \text{Enc}, \text{Dec})$ esquema de cifrado **CPA secure**.

$\pi_M(\text{Gen}_M, \text{Mac}, \text{Vrfy})$ esquema de **MAC seguro**, con etiquetas únicas ($\forall k, \forall m$, hay un único valor t para el cual $\text{vrfy}(m, t) = 1$.)

- **Gen'**: $k_1 \leftarrow \text{Gen}_E(1^n)$, $k_2 \leftarrow \text{Gen}_M(1^n)$,
- **Enc'**: $c \leftarrow \text{Enc}_{k_1}(m)$ y $t \leftarrow \text{Mac}_{k_2}(c)$. Emitir $\langle c, t \rangle$
- **Dec'**: Primero chequear si $\text{vrfy}_{k_2}(c, t) = 1$.
Si verifica, emitir como salida $\text{Dec}_{k_1}(c)$. Sino fallo.

Las claves k_1 y k_2 deben ser independientes

Este esquema pasa el experimento de indistinguibilidad ante CCA.

(Ver teorema 4.20.)

La idea es mostrar que la desenscripción que podría dar el oráculo no sirve para nada al adversario.

Ejercicio:

Considerar:

- F una función pseudoaleatoria biyectiva fuerte
- $Enc_k(m) = F_k(m || r)$
- $Mac_k(c) = F_k^{-1}(c)$ (pasa Mac Forge)

Probar que el esquema haciendo Cífrar y luego autenticar no es resistente a CCA (por tener claves iguales).